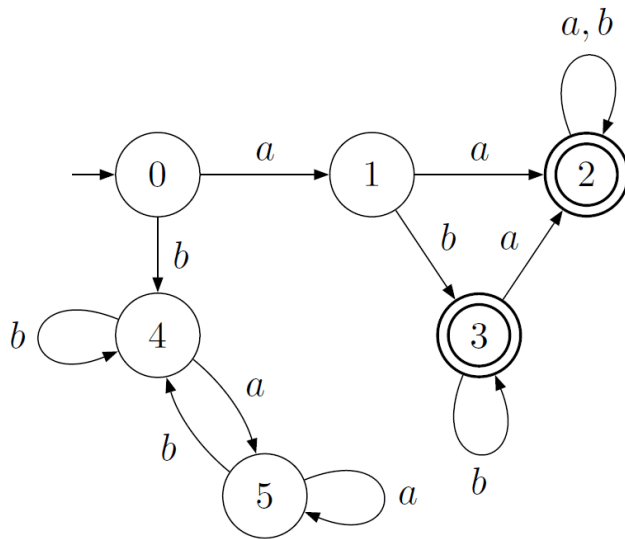




G52LAC

Languages and Computation

Lecture 1: Administrative Details and Introduction

Tomas Maul
School of Computer Science
University of Nottingham Malaysia Campus

Outline

- Contact details.
- Timetable.
- Pre-requisites.
- Overview of Lectures.
- Assessment.
- Reading.

Contact Details

- ◀ Tomas Maul
- ◀ Room BB65
- ◀ E-mail: Tomas.Maul@nottingham.edu.my
- ◀ Consultation:
 - ◀ **Times.** During lectures; during breaks; impromptu or planned meetings (outside lecture times). Impromptu meetings need to be very short. For longer meetings, please send me an e-mail in order to book an appointment.
 - ◀ Default consultation time. Wednesdays from 1pm to 2pm.
 - ◀ **Methods.** Email, Teams messages, Moodle Forum, meetings.
- ◀ Web page:
 - ◀ [Moodle](#).
- ◀ Main areas of research: Neural Networks; Deep Learning; Computational Neuroscience; Image Processing; Computer Vision.

Timetable

- ◀ Two hours of lectures per week:
 - Day: Tuesday.
 - Time: 10:00 – 12:00.
 - Venue: F3C04.
- ◀ One hour of tutorials per week:
 - Not in the first week.
 - Time: 12:00-13:00.
 - Venue: F3C04.
 - The slot can sometimes be used for other purposes, e.g.: lecturing and/or assessed quizzes.

Assessment

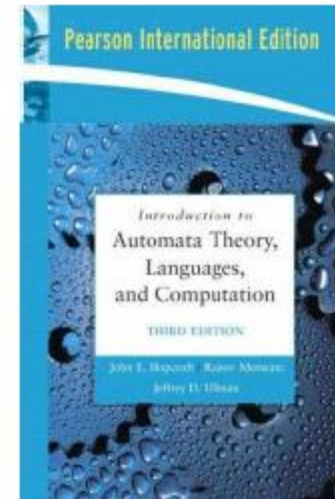
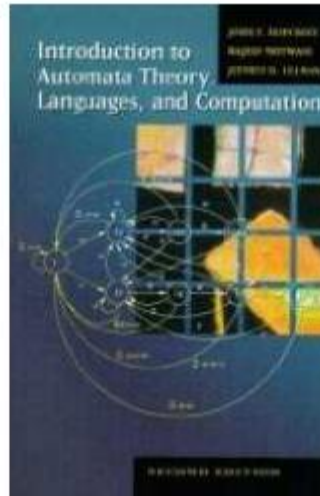
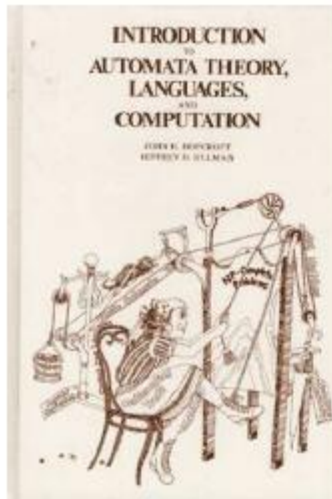
- Assessment:
 - Coursework (25%). Details to be provided soon.
Based on live quizzes.
 - Exams (75%).
- Practice ...

Let's have a look at Moodle

Pre-requisites

- None, but the following help:
 - Mathematics for Computer Scientists
 - Programming and Algorithms
 - Programming Paradigms

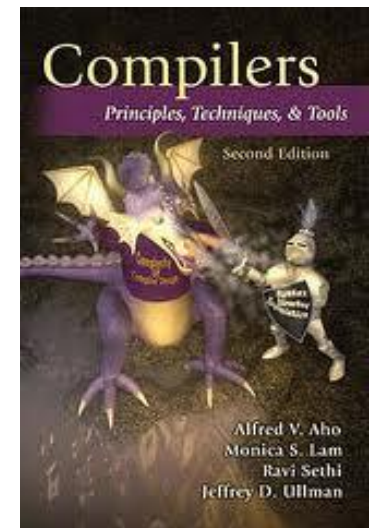
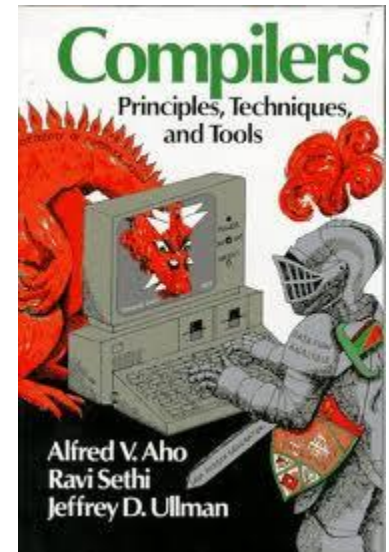
Reading



- Main reference:
Introduction to Automata Theory, Languages, and Computation; 2nd or 3rd edition; Hopcroft, Motwani and Ullman; (1979, 2000 & 2006).
- *G52MAL Lecture Notes*, Altenkirch and Nilsson, 2007.
Available via the G52MAL module web-page.
- Your own notes from the lectures!

Compilers

- Topics covered in Languages and Computation (LAC) have important applications in the area of compiler construction.
- In fact, LAC feeds directly into next year's Compilers module.
- If you are curious you might want to check out “The Dragon Book”: Compilers — Principles, Techniques, and Tools; Aho, Sethi & Ullman; 1986.



Aims of the Module

- To familiarize you with key Computer Science concepts in central areas such as:
 - Automata Theory
 - Formal Languages
 - Models of Computation
- This includes:
 - the equivalence between machine types and language types;
 - the specification of formal languages by grammars and other notations;
 - the relevance of machines that process strings from an alphabet as models of computation;
 - and the limits on what is computable by any machine.

Content

- Mathematical models of computation, such as:
 - Finite Automata
 - Pushdown Automata
 - Turing Machines
- How to specify formal languages?
 - Regular Expressions
 - Context Free Grammars
 - Context Sensitive Grammars
- The relationship between the two.

Overview of Lectures

Refer to the module's Moodle page.

Languages and Computation (COMP2040 UNMC) (SPM1 24-25)

Module

Settings

Participants

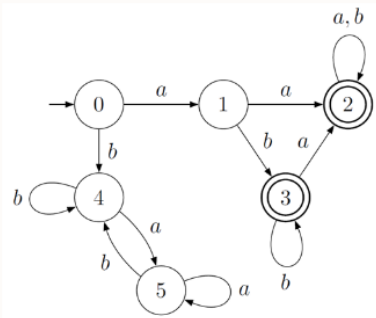
Grades

Reports

More ▾

▾ General Information

[Collapse all](#)



Why Study Automata Theory?

- Finite automata are a useful model for important kinds of hardware and software:
 - Software for designing and checking digital circuits.
 - Lexical analysers and parsers for compilers.
 - Finding words and patterns in large bodies of text (e.g. in web pages).
 - Verification of systems with finite numbers of states (e.g. communication protocols).
 - Even modeling for robots.

Why Study Automata Theory?

- The study of Automata and Formal Languages are intimately connected. Methods for specifying formal languages are very important in many areas of Computer Science, for example:
 - **Context Free Grammars** are very useful when designing software that processes data with recursive structure, like the parser in a compiler.
 - **Regular Expressions** are very useful for specifying lexical aspects of programming languages and search patterns.

Example: Regular Expressions

- Suppose you need to locate a piece of text in a directory containing a large number of files of various kinds. You recall only that the text mentions the year 19-something.
- The following UNIX-command will do the trick:
 - *grep "19[0-9][0-9]" *.txt*
- The command involves TWO (extended) **regular expressions**.



<http://en.wikipedia.org/wiki/Grep>

Example: Regular Expressions

The result is a list of names of files containing text matching the pattern, together with the matching text lines:

history.txt: In 1933 it became
notes.txt: later on, around 1995,

How can we write software that quickly searches for this kind of patterns?

Not as easy as it seems!

You will learn the basic answer in this module.

Example: Regular Expressions



<http://www.regexr.com/>



http://en.wikipedia.org/wiki/Automata_theory

`/[A|a]utomat(\S)+\sc/g`

RegExr v2.0

by gskinner RegExr v1 GitHub Tutorial

Character classes

- not digit `\D`
- whitespace `\s`
- not whitespace `\S`
- character set `[ABC]`
- negated set `[^ABC]`
- range `[A-Z]`

Character classes match a character from a specific set. There are a number of predefined character classes and you can also define your own sets.

Expression

`/[A|a]utomat(\S)+\sc/g` 2 matches

Text

Automata theory is the study of abstract machines and automata, as well as the computational problems that can be solved using them. It is a theory in theoretical computer science, under Discrete mathematics (a section of Mathematics and also of Computer Science). **Automata** comes from the Greek word αὐτόματα meaning "self-acting". Automata Theory is the study of self-operating virtual machines to help in logical understanding of input and output process, without or with intermediate stage(s) of computation (or any function / process).

The figure at right illustrates a finite state machine, which belongs to one well-known variety of automaton. This **automaton** consists of states (represented in the figure by circles), and transitions (represented by arrows). As the automaton sees a symbol of input, it makes a transition (or jump) to another state, according to its transition function (which takes the current state and the recent symbol as its inputs).

Why Study Automata Theory?

- Automata are essential for the study of the **limits of computation**. Two issues:
 - What can a computer do at all? (**Decidability**)
 - What can a computer do efficiently? (**Tractability**)

Example: The Halting Problem

Given a program and an input, can we determine whether the program will eventually halt?

```
int a = 0;  
While (true) {  
    a++; a--;  
}
```

Will not halt

```
int a = 0;  
print(a);
```

Will halt

For the examples above, it is easy to say/decide whether the programs will halt or not.

Can we write a program that will **always** be able to tell us (i.e. decide) whether ANY arbitrary program with ANY input, will terminate (or halt) or not?

Example: The Halting Problem

Consider the following program:

```
while ( $n > 1$ ) {  
    if even ( $n$ ) {  
         $n = n / 2$ ;  
    } else {  
         $n = n * 3 + 1$ ;  
    }  
}
```

Does it terminate for all values of $n \geq 1$? (where $n \in \mathbb{N}$)



... with $n = 7$

Example: The Halting Problem

For example, say we start with $n = 7$:

7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1

In fact, for all numbers that have been tried (**a lot!**), it does terminate . . .

. . . but no one has ever been able to **prove** that it **always** terminates!

Example: The Halting Problem

Thus the following important decidability result should perhaps not come as a total surprise:

It is impossible to write a program that decides if another, arbitrary program with any input terminates (halts) or not.

What might be surprising is that it **is** possible to **prove** such a result (i.e. **it is possible to prove that it is impossible to write a program that can always decide if any other, arbitrary program with any input halts or not**). This was first done by **Alan Turing**.

It was not possible to do the above without **formalizing the notion of program and computation**. Alan Turing formalized our notion of program and computation (i.e. Turing Machine). This constituted some of the necessary groundwork for the above proof.

Alan Turing

Alan Turing (1912–1954):

- British mathematician.
- Introduced an abstract model of computation, **Turing Machines**, to give a precise definition of what problems can be solved by a computer.
- Also famous for being instrumental in the success of British code-breaking efforts during the Second World War.



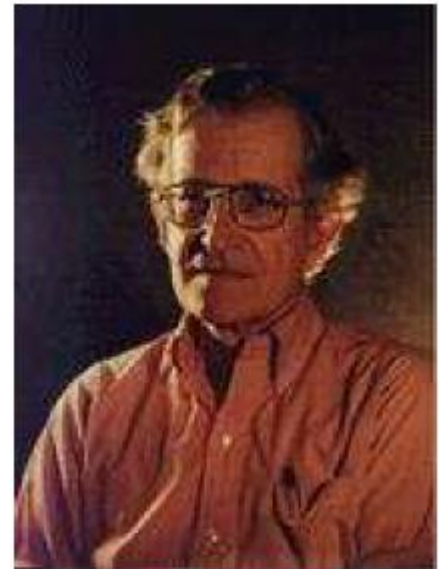
<https://www.youtube.com/watch?v=nuPZUUED5uk>



Noam Chomsky

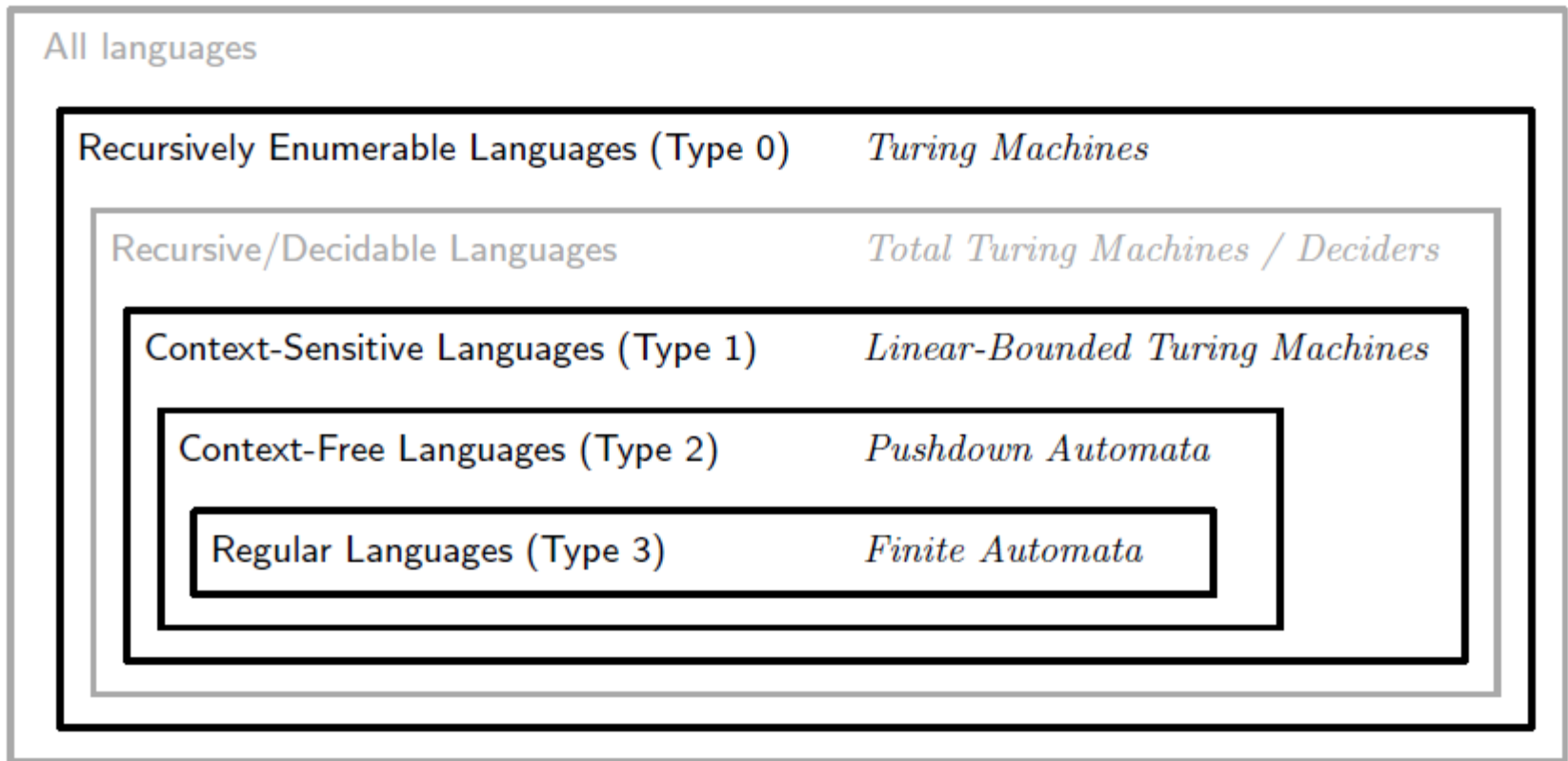
Noam Chomsky (1928–):

- American linguist.
- Introduced **Context Free Grammars** in an attempt to describe natural languages formally.
- Introduced the **Chomsky Hierarchy** which classifies grammars and languages, and their descriptive power.
- Also famous for his political activism.



<https://www.youtube.com/watch?v=Zg1bHzBoggk>

The Chomsky Hierarchy



Recommended Reading

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 1–5.
- G52MAL Lecture Notes, pages 2–5. [go to the module web page]

Acknowledgements

- Venanzio Capretta.
- Thorsten Altenkirch.
- Neil Sculthorpe.
- Henrik Nilsson.
- V.P. Kallimani.